

Ciclo for

Ciclo for

Come usare il ciclo for in C++ per iterazioni controllate.

A cosa serve il ciclo **for** ?

Immagina di dover stampare i numeri da 1 a 100. Scriveresti 100 righe di codice? Certo che no.

Il ciclo **for** ti permette di ripetere un blocco di codice un numero preciso di volte. Scrivi il codice una volta sola, e il programma lo esegue quante volte vuoi.

La struttura del ciclo **for**

```
for (inizio; condizione; aggiornamento) {  
    // codice da ripetere  
}
```

Un esempio concreto:

```
for (int i = 0; i < 5; i++) {  
    cout << i << endl;  
}
```

Output:

0
1
2
3
4

Le tre parti del **for**

```
for (int i = 0; i < 5; i++) {  
  //   ^^^^^^^   ^^^^^   ^^^  
  //  inizio     cond.    aggiorn.
```

1. **Inizio** (`int i = 0`): viene eseguita una sola volta, all'inizio. Crea il "contatore" del ciclo.
2. **Condizione** (`i < 5`): viene controllata prima di ogni ripetizione. Se è falsa, il ciclo si ferma.
3. **Aggiornamento** (`i++`): viene eseguito alla fine di ogni ripetizione. Di solito aumenta il contatore.

Pensa al ciclo `for` come a un cronometro: parti da zero, vai avanti di uno in uno, ti fermi quando arrivi al limite.

Contare in avanti

```
// Da 1 a 10  
for (int i = 1; i <= 10; i++) {  
    cout << i << " ";  
}  
// Output: 1 2 3 4 5 6 7 8 9 10
```

Contare all'indietro

```
// Da 10 a 1 – basta cambiare inizio, condizione e aggiornamento
for (int i = 10; i >= 1; i--) {
    cout << i << " ";
}
// Output: 10 9 8 7 6 5 4 3 2 1
```

Passo diverso da 1

Non devi necessariamente aumentare di uno alla volta:

```
// Solo i numeri pari da 0 a 10
for (int i = 0; i <= 10; i += 2) {
    cout << i << " ";
}
// Output: 0 2 4 6 8 10

// Multipli di 5
for (int i = 5; i <= 25; i += 5) {
    cout << i << " ";
}
// Output: 5 10 15 20 25
```

Cicli annidati

Puoi mettere un ciclo dentro un altro. Utile per lavorare con griglie, tabelle o schemi:

```
// Per ogni riga (i) eseguiamo tutto il ciclo delle colonne (j)
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        cout << i * j << "\t"; // \t è il tabulatore (allinea le colonne)
    }
    cout << endl; // vai a capo dopo ogni riga
}
```

Output (tabellina pitagorica 3x3):

```
1  2  3
2  4  6
3  6  9
```

Il for su una collezione (range-based for)

In C++ moderno puoi usare una forma più semplice di `for` per scorrere gli elementi di una lista o di una stringa, senza gestire manualmente l'indice:

```
// Scorre ogni carattere della stringa nome
string nome = "Alice";
for (char c : nome) {
    cout << c << " ";
}
// Output: A l i c e

// Scorre ogni numero dell'array
int numeri[] = {10, 20, 30, 40, 50};
for (int n : numeri) {
    cout << n << " ";
}
// Output: 10 20 30 40 50
```

La sintassi è: `for (tipo elemento : collezione)`. Si legge "per ogni elemento nella collezione".

Usare `auto` per non specificare il tipo

Se non vuoi scrivere il tipo dell'elemento, puoi usare `auto` e lasciare che il compilatore lo capisca da solo:

```
int numeri[] = {1, 2, 3, 4, 5};
for (auto n : numeri) {
    cout << n * 2 << " "; // stampa il doppio di ogni numero
}
// Output: 2 4 6 8 10
```

Esempio pratico: tabella delle potenze

```
#include <iostream>
using namespace std;

int main() {
    // Intestazione della tabella
    cout << "n\tn2\tn3" << endl;
    cout << "----\t----\t----" << endl;

    // Calcoliamo n2, n3 per ogni numero da 1 a 10
    for (int n = 1; n <= 10; n++) {
        cout << n << "\t" << n*n << "\t" << n*n*n << endl;
    }

    return 0;
}
```

Output:

n	n ²	n ³
----	----	----
1	1	1
2	4	8
3	9	27
4	16	64
5	25	125
...		